

README.en

SoDam-Reverse-Eng — AI Code Analysis Plugin for Claude Code

Completely new to computers or AI? That's fine — follow this document step by step.

A Claude Code plugin that **explains your own (or authorized) code in plain language** using AI.

 **Honest Promise:** Crack guides, DRM bypass, authentication bypass, license bypass, and credential extraction are **refused no matter what** — this tool is exclusively for defense, education, and analysis of code you own.

Table of Contents

1. [What Is This?](#)
2. [What Does It Do?](#)
3. [What Does It Refuse?](#)
4. [SoDam Family Synergy.](#)
5. [Prerequisites](#)
6. [Installation](#)
7. [Commands](#)
8. [Analysis Workflow](#)
9. [Security · Data Flow](#)
10. [Architecture · File Structure](#)
11. [File · Document Locations](#)
12. [Troubleshooting · FAQ](#)
13. [License · Copyright · Commercial Use](#)

14. Contact · Contributing

1. What Is This?

A **plugin** is a small add-on that gives an existing program new capabilities — similar to installing an app on your smartphone.

SoDam-Reverse-Eng is a plugin that adds “code analysis” capability to **Claude Code** (an AI tool).

In Simple Terms

- You ask the AI: “What does this code do?”
- The AI reads the code and **explains it in plain language**
- The result is organized into a **standard report**
- Dangerous requests (cracking, bypass, etc.) are **automatically refused**

Who Is It For?

- When you cannot remember how your old code worked
 - When you need to understand someone else’s code (with permission)
 - When you are wondering “Why does this function behave this way?”
 - When you want to find security vulnerabilities in your own code
-

2. What Does It Do?

Feature	Description
 Code Explanation	“What does this function do?” → explained in plain language
 Standard Report	Summary · per-function explanation · source location · uncertainties · next steps
 3-Layer Safety	AI refusal (Layer 1) + real-time block (Layer 2) + integrity check (Layer 3)
 Auto-Masking	Passwords, API keys, tokens shown as •••• in reports

Feature	Description
 Full Korean UI	All messages and errors displayed in Korean
 Local Storage Only	Results saved only to your computer (no external upload)

3. What Does It Refuse? (Safety Boundary)

Request	Result	Reason
“Explain how this code works”	 Proceed (after consent)	Allowed
“Find bugs for me”	 Proceed (after consent)	Allowed
“Where are the security vulnerabilities?”	 Proceed (after consent)	Allowed
“Tell me how to crack this”	 Refused	Absolutely prohibited
“Write authentication bypass code”	 Refused	Absolutely prohibited
“Extract tokens/passwords”	 Refused	Absolutely prohibited
“How do I remove DRM?”	 Refused	Absolutely prohibited
“Bypass the license”	 Refused	Absolutely prohibited
API key found during analysis	(masked)	Privacy protection

4. SoDam Family Synergy

SoDam-Reverse is the **youngest** of 6 sibling plugins. Installing siblings makes it more powerful.

Recommended installation order: Harness → Loop → Context → Agentic → Prompt → **Reverse (this one)**

Sibling Plugin	Role	With Reverse
SoDam-Harness	Safety belt · backup	Shares RE patterns → one hook covers both (no duplication)
SoDam-Loop	Safe repetition control	Keeps repeated analysis loops safe
SoDam-Context	CLAUDE.md health check	Auto-detects scope violations
SoDam-Agentic	Planning · easy review	Re-reads reports for non-developer readability
SoDam-Prompt	Improves natural language requests	Better “analyze this” quality

 **Important:** Starting Claude Code from `c:\Users\name` (home folder) causes Harness and Loop guards to block even safe operations. Always start from a **project folder** (`D:\my-project\` etc.).

If Harness is installed — run once:

```
node scripts/re-inject-harness.mjs
```

→ RE danger patterns are added to Harness safety rules and the hook is shared (no duplication).

5. Prerequisites (Free, 2 Items)

Item 1: Node.js (Version 18 or higher)

Node.js is the “runtime engine” needed for the safety guards to work.

Check if you already have it: 1. Press  **Windows key** 2. Type `powershell` → click **Windows PowerShell** 3. In the black window, type and press **Enter**:
`node --version` 4. If you see a number like `v18.` or higher → **OK** (e.g., `v20.19.0`, `v22.14.0`)

If not installed (or lower than v18): 1. Open a web browser (Chrome, Edge, etc.) → go to **nodejs.org** 2. Click the green “**LTS**” button → download the .msi file 3. Double-click the downloaded file → keep clicking “**Next**” → install 4. **Close PowerShell completely** → reopen it 5. Run `node --version` again to confirm

💡 You may need to reboot your computer for it to take effect.

Item 2: Claude Code

If you are reading this inside Claude Code, **you already have it**. If not, install Claude Code from the Anthropic official website.

6. Installation

📌 **Detailed step-by-step guide:** See [GUIDE.en.md](#) Chapter 2

⚠️ Read Before Installing — Most Common Failure Cause

#1 reason commands do not appear: Claude Code was started in the wrong folder

Claude Code reads commands **based on the folder it was started in**. Typing `cd other-folder` inside the chat does NOT reload commands.

Correct sequence: 1. Open PowerShell (black window) 2. Type `cd D:\my-project-folder` (navigate to your project folder) 3. Type `claude` to launch Claude Code

❌ Starting from the home folder (`C:\Users\name`) means `/re-start` will not appear.

Step 1: Register the Marketplace

Type one of the following in the Claude Code chat:

Option A — From GitHub (recommended after public release):

```
/plugin marketplace add sodam-ai/SoDam-Reverse-Eng
```

Option B — From local folder (current development phase):

```
/plugin marketplace add D:\AI_Dev_Work\2026y\26y_06m_30d_SoDam-Reverse-Eng
```

💡 Replace the path in Option B with the actual plugin folder path on your machine.

Step 2: Install the Plugin

After registering the marketplace, type:

```
/plugin install sodam-reverse@sodamreverse-marketplace
```

Or type `/plugin` → menu → **Browse marketplaces** → **sodam-reverse** → **Install**.

Step 3: Full Restart (Required!)

Plugins are **only loaded when Claude Code starts**. You must fully exit and restart.

1. Type `/exit` in the chat → Enter
2. **Completely close** the terminal window
3. Open a new PowerShell window
4. **Start from your project folder:**

```
cd D:\my-project-folder
claude
```

Step 4: Verify Installation

Once Claude Code opens, type `/re-p` — autocomplete should show:

```
/re-ping    (sodam-reverse) ← diagnostic
/re-start   (sodam-reverse) ← start analysis
```

The `(sodam-reverse)` tag confirms the plugin loaded correctly.

Run the diagnostic command:

```
/re-ping
```

→ If you get "Pong! /re-ping is working." — installation is complete.

Step 5: Verify the 3-Layer Safety (Required!)

```
/re-selftest
```

Expected result:    All three green

If any are not green → See [GUIDE.en.md](#) Chapter 2

Step 6: Register Integrity Hash (Activate Layer 3)

Copy the SHA-256 hash from the selftest output → Paste it as the value for `hooks/re-deny-guard.mjs` in `references/integrity.json` → Save → Run `/re-selftest` again → Confirm Layer 3 

7. Commands

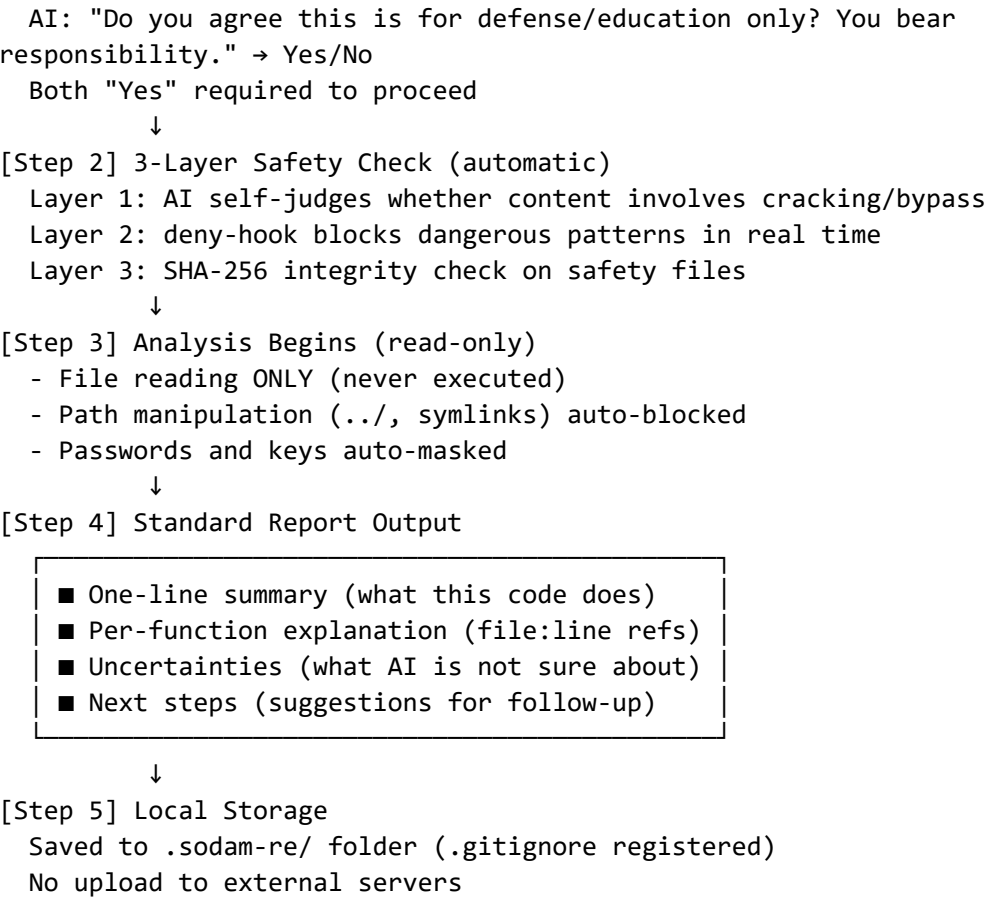
Command	When to Use	Example
<code>/re-ping</code>	Verify installation · diagnostics (test this first)	<code>/re-ping</code>
<code>/re-start [file-path]</code>	Start a new analysis	<code>/re-start src/login.js</code>
<code>/re-report</code>	View the last analysis report again	<code>/re-report</code>
<code>/re-selftest</code>	Check the 3-layer safety system	<code>/re-selftest</code>

Upcoming commands (after Phase 2·3):

Command	Status	Description
<code>/re-android [APK-path]</code>	 Phase 2	Android app analysis
<code>/re-binary [file-path]</code>	 Phase 3	Binary/executable analysis

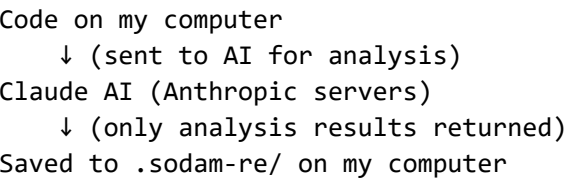
8. Analysis Workflow

User: `/re-start my-code/login.js`
↓
[Step 1] Consent Gate
AI: "Is this code yours or do you have permission to analyze it?" → Yes/No



9. Security · Data Flow

Where Does My Code Data Go?



- **Sent to AI:** Code content (required for analysis — Anthropic API Terms apply)
- **NOT sent externally:** Analysis results, consent records, block logs
- **Masked before saving:** Passwords, keys, tokens found in code

3-Layer Safety Architecture

Layer	Name	Role	File
Layer 1	AI Refusal Rules	AI itself refuses to output	skills/re-router/SKILL.md

Layer	Name	Role	File
Layer 2	deny-hook	crack/bypass content	hooks/re-deny-guard.mjs
		Real-time blocking of dangerous keyword patterns	
Layer 3	Integrity Check	Detects tampering via SHA-256 hashing	hooks/_selftest.mjs

fail-closed principle: If a hook encounters an error, analysis is **immediately halted** — not passed through.

Analysis Result Storage

```
[plugin-folder]/
├── .sodam-re/                ← Results folder (auto-created)
│   ├── reports/              ← Report files
│   ├── consent/              ← Consent records (con-TIMESTAMP.json)
│   └── safety-log.jsonl       ← Block event log (content is hashed)
```

The .sodam-re/ folder is registered in .gitignore — **never uploaded to Git**.

10. Architecture • File Structure

```
SoDam-Reverse-Eng/
├── .claude-plugin/            ← Claude Code plugin declaration
│   ├── plugin.json
│   └── marketplace.json
├── commands/                  ← Command definitions
│   ├── re-ping.md             ← /re-ping (diagnostic)
│   ├── re-start.md            ← /re-start (analysis start)
│   ├── re-report.md           ← /re-report
│   ├── re-selftest.md         ← /re-selftest
│   ├── re-android.md          ← Phase 2 (stub)
│   └── re-binary.md           ← Phase 3 (stub)
├── skills/                    ← AI analysis logic
│   └── re-router/             ← Layer 1 safety rules + request routing
```

└─ re-analyze-mycode/	← Source code analysis
└─ re-report/	← Report generation
└─ re-analyze-android/	← Phase 2 (stub)
└─ re-analyze-binary/	← Phase 3 (stub)
└─ hooks/	← Safety layers 2 and 3
└─ re-deny-guard.mjs	← Layer 2: real-time danger blocking
└─ _selftest.mjs	← Layer 3: SHA-256 integrity check
└─ hooks.json	← Hook configuration
└─ references/	← Data and rule files
└─ deny-corpus.json	← 43 dangerous patterns
└─ mask-patterns.json	← 10 masking rules
└─ trust-catalog.md	← 15 trusted tool entries
└─ report-template.md	← Standard report template
└─ integrity.json	← SHA-256 hash store
└─ scripts/	← Utility scripts
└─ re-inject-harness.mjs	← Harness synergy setup
└─ re-inject-context.mjs	← Context synergy setup
└─ check-family.mjs	← 6-sibling status check
└─ check-trust-freshness.mjs	
└─ samples/	← Example files for testing
└─ safe-login.js	
└─ deny-demo.txt	
└─ .sodam-re/	← Analysis results (auto-created,
└─ .gitignore)	
└─ README.md	← Korean overview
└─ README.en.md	← This file (English overview)
└─ GUIDE.md	← Korean detailed guide
└─ GUIDE.en.md	← English detailed guide
└─ TROUBLESHOOTING.md	← Error resolution guide
└─ LICENSE	← Apache-2.0 full text
└─ NOTICE	← Copyright notice

11. File · Document Locations

File / Document	Location	Purpose
Korean README	README.md	Korean overview
English README	README.en.md	This file
Korean Guide	GUIDE.md	Comprehensive Korean guide
English Guide	GUIDE.en.md	Comprehensive English guide

File / Document	Location	Purpose
Error Resolution	TROUBLESHOOTING.md	Full troubleshooting reference
Safety File Code	SETUP_BLOCKED_FILES.md	Manual setup file contents
License Full Text	LICENSE	Apache-2.0 full text
Copyright Notice	NOTICE	Third-party notices
Dev Progress	CHECKPOINT.md	Developer milestones
Danger Pattern DB	references/deny-corpus.json	43 blocked patterns
Masking Patterns	references/mask-patterns.json	10 masking rules
Trusted Tool List	references/trust-catalog.md	15 tools with trust ratings
Report Template	references/report-template.md	Report format definition
Integrity Hashes	references/integrity.json	SHA-256 hash store
Family Status	scripts/check-family.mjs	6-sibling diagnostic script

12. Troubleshooting · FAQ

Q1. /re-start does not appear or shows “Unknown command” (most common)

Cause: Claude Code was launched from the home folder (C:\Users\name) or you used `cd` inside the chat (commands do not reload on `cd`).

Fix: 1. Fully close Claude Code 2. Open a new PowerShell window 3.

Navigate to your project folder, then launch: `powershell cd D:\my-project-folder` `claude` 4. Type `/re-ping` → confirm "Pong!" response

Q2. What is /re-ping?

A diagnostic command to verify the plugin loaded. If you get "Pong! /re-ping is working.", the plugin is loaded correctly. If /re-ping fails, /re-start will also fail.

Q3. “Node.js not found” error

nodejs.org → install **LTS** → reboot → reopen terminal → `node --version`

Q4. Normal operations keep getting blocked

You are likely running Claude Code from `C:\Users\name` (home folder).

```
cd D:\my-project-folder
claude
```

Restart Claude Code from a **project folder**.

Q5. The consent question keeps prompting

Answer clearly with yes, Yes, or I agree. Vague answers like “I think so” or “probably” are not treated as consent.

Q6. Crack/bypass request was refused

This is correct behavior. This tool is exclusively for defense, education, and analysis of code you own.

Q7. /re-selftest shows some items as ❌

Check that all 5 files from `SETUP_BLOCKED_FILES.md` exist → Recreate any missing files → Fully restart Claude Code → Run /re-selftest again

SHA-256 mismatch:

```
node hooks/_selftest.mjs
```

Copy output hash → save to `references/integrity.json` → rerun

Q8. Passwords appear in plain text in the report

This is a bug. The pattern is not in `references/mask-patterns.json`. Do not share that report. File a GitHub Issue.

Q9. Analysis is taking too long

Analyze one file at a time:

```
/re-start src/auth.js
```

File-by-file analysis is much faster than folder-wide.

Q10. Harness synergy is not connecting

```
node scripts/re-inject-harness.mjs
```

“N RE rules injected” message means success.

Q11. More detailed error resolution

→ [GUIDE.en.md](#) — 25 FAQs and detailed guide → [TROUBLESHOOTING.md](#)
— Full error pattern reference

13. License · Copyright · Commercial Use

Applied License

Apache License 2.0 · © 2026 SoDam AI Studio

Explicitly Permitted

Action	Permitted?
Personal use	 Permitted
Modification	 Permitted (must document changes)

Action	Permitted?
Copying and forking	✓ Permitted
Redistribution	✓ Permitted (must include license and copyright notice)
Commercial use	✓ Permitted
Patent use	✓ Permitted

Required Actions (Obligations)

Obligation	Details
Preserve license notice	Must include the LICENSE file
Preserve copyright notice	Keep © 2026 SoDam AI Studio
Document changes	Must state what was modified if you change the code
Include NOTICE file	Must distribute NOTICE file alongside

Not Permitted

Action	Notes
Unauthorized use of “SoDam” trademark	Cannot use as your own product/service trademark without permission
Warranty claims	Provided AS-IS — no warranty of quality or fitness
Liability claims	Disclaimer applies

Third-Party Trademark Notice

- “Claude”, “Claude Code”, “Anthropic” — Trademarks of Anthropic PBC
- “IDA Pro” — Trademark of Hex-Rays SA (Phase 3 optional feature)
- The above are **not affiliated with or endorsed by** this product

Commercial Use Checklist

Item	What to Check
This tool itself	Apache-2.0 allows commercial use
Analyzed software	Check that software's EULA and license separately
Claude API usage	Anthropic Terms of Service apply separately
IDA Pro (Phase 3, optional)	Requires separate commercial license purchase

Disclaimer

- Provided “**AS-IS**”
- **No warranty** of quality, performance, or fitness for a particular purpose
- **User bears responsibility** for the results of use
- No security tool can guarantee 100% safety

Full license text: [LICENSE](#) · **Copyright notice:** [NOTICE](#) **Detailed license explanation:** See [GUIDE.en.md](#) Chapter 12

14. Contact · Contributing

- **Bug reports and feature requests:** GitHub Issues (active after public release)
- **Email:** startmxk@gmail.com
- **How to contribute:** Fork → Modify → Pull Request (Apache-2.0 license applies)

Korean version: [README.md](#) *Detailed English guide:* [GUIDE.en.md](#) *Error resolution:* [TROUBLESHOOTING.md](#)